Scikit-learn and Tour of Classifiers

K-Nearest Neighbours (K-NN)



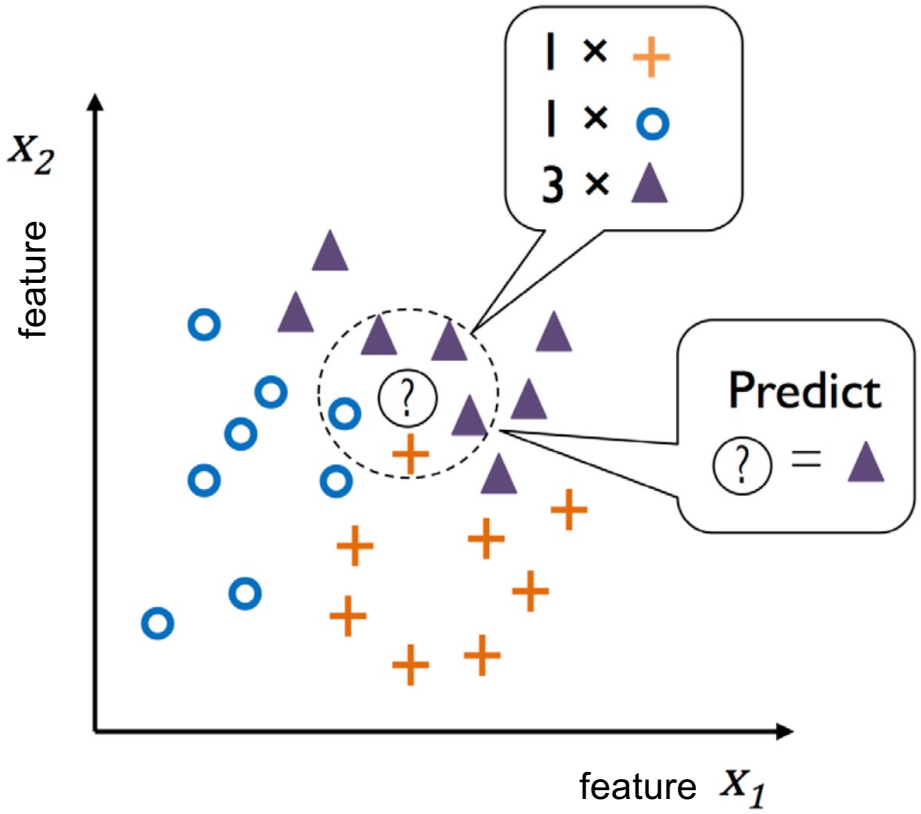
K-Nearest Neighbours

- Fundamentally different algorithm from what we learned so far
- Does not learn a discriminative function
- Memorizes the training data set and makes predictions based on that

K-Nearest Neighbours – Algorithm

1. Choose the parameter ”***k***” and a **distance metric**
2. Find the k-nearest neighbours of a data record that we want to classify
3. Assign the class label by a majority vote
   - *Tie break*: Take the class of the closest neighbour, and if still tied use the lowest label

K-Nearest Neighbours – Algorithm



Example: 5-nearest neighbours of a new data record *(?)* that we want to classify

Figure from Python Machine Learning (Raschka & Mirjalili)

K-Nearest Neighbours – Distance metric

- A **distance metric** measures the **distance** between two samples
- **Should fit the type of data**
- Common for real numbers: Euclidean distance

$$d\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \sqrt[p]{\sum_k \left|x_k^{(i)} - x_k^{(j)}\right|^p}$$

Minkowski distance (used in scikit-learn K-NN)

p = 1

$$\sum_k \left|x_k^{(i)} - x_k^{(j)}\right|$$

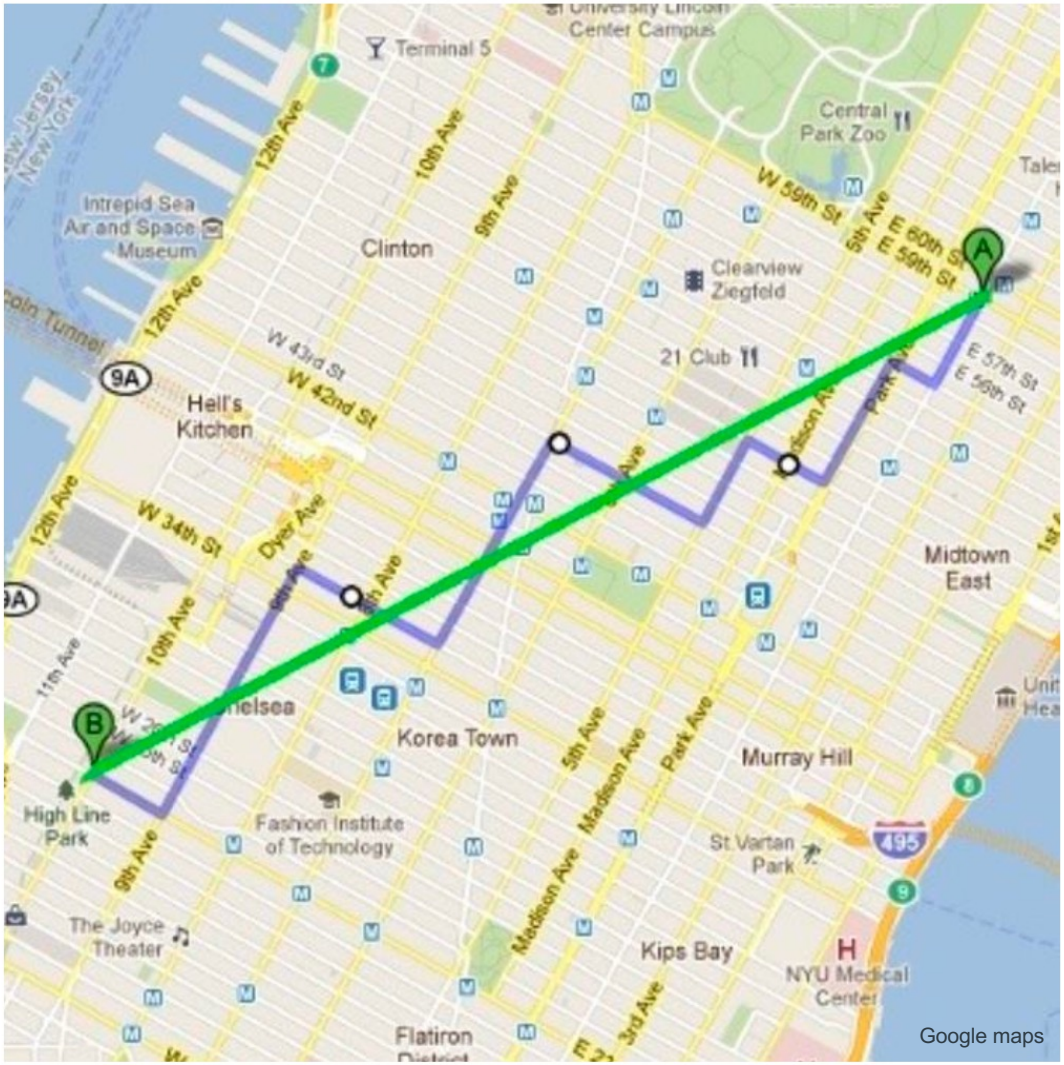
Manhattan distance

p = 2

$$d\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \sqrt{\sum_k \left|x_k^{(i)} - x_k^{(j)}\right|^2}$$

Euclidean distance

- Euclidean distance
- Manhattan distance





K-Nearest Neighbours

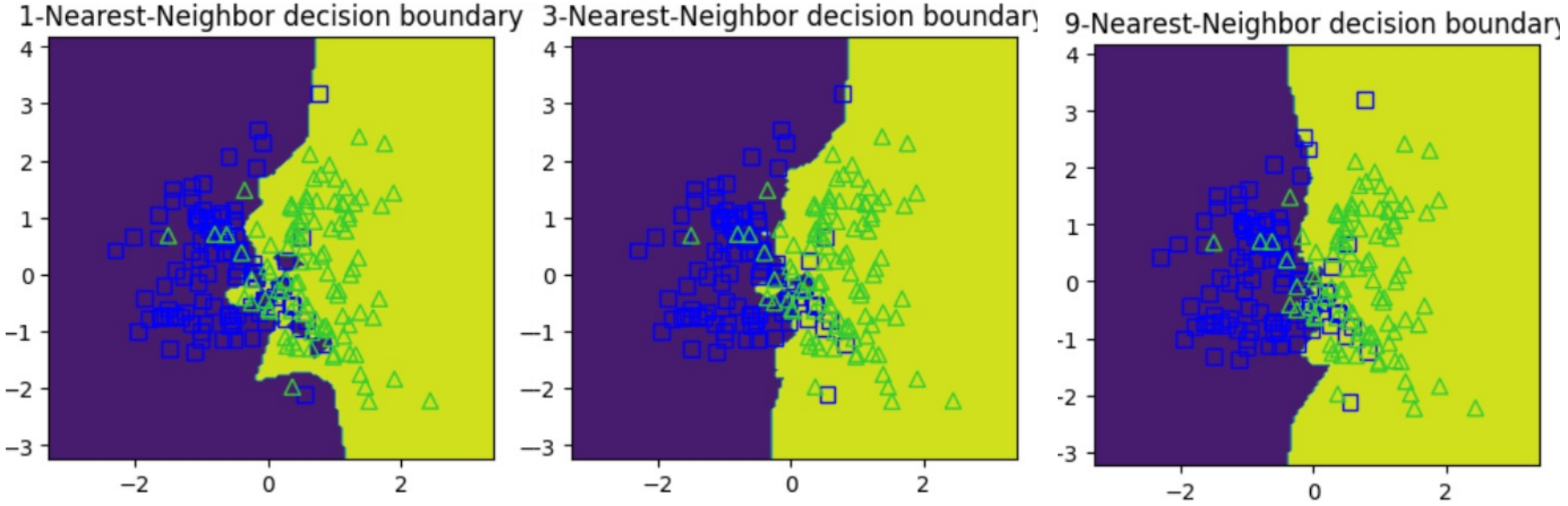
- Tuning ***k*** (and the distance metric) is crucial to find a good balance between overfitting and underfitting
  - Lower ***k*** means that the model is more complex and has higher variance
  - Higher ***k*** means less variance (consulting more neighbours), but higher bias (far away neighbours might be consulted)
  - Higher ***k*** means that the prediction becomes more costly

K-Nearest Neighbours

`03_knn.ipynb`

`03_knn_hyperparameters_validation_set.ipynb`

- Code examples, testing the accuracy on the test/training set for different values of ***k***



Parametric versus non-parametric models

Parametric versus non-parametric models

- **Parametric models**
  - Learn **parametrized** discriminative functionals (fixed sets of parameters)
  - Predict/classify new points **without the need for** the training data
  - Examples: Perceptron, Adaline, Linear Regression, Logistic Regression, (*linear* SVM)
- **Non-parametric models**
  - No fixed set of parameters, the number of parameters **changes with the amount of training data**
  - The training data (at least a subset) is needed for prediction
  - Examples: K-NN, most kernel machines (Kernel-SVM, Kernel-Perceptron, …)

Some advantages and disadvantages of a memory-based non-parametric approach like K-NN

- Advantages:
  - Low (or zero) cost of adaption to new training samples
  - Good predictive vs. computational performance for small to medium-sized data sets
- Disadvantages:
  - Storage and prediction costs grow with the number of samples (the prediction cost can be lowered by using efficient data structures, e.g. k-d trees for K-NN)
  - **Curse of dimensionality** → for a high number of features => prone to overfitting, no other training sample may be informative due to a sparse feature space population resulting from a high number of dimensions (number of features)

Concluding our Tour of Classifiers

Tour of Classifiers



- Learned about a number of popular classifiers for tackling linear and nonlinear classification problems; some insights into the corresponding algorithms, and how to use them in scikit-learn
- **Logistic regression:** Allows for predicting the probability of a particular event
- **Support vector machines:** Powerful linear models; extended to non-linear problems using the kernel trick; several parameters need to be tuned to make good predictions
- **Decision trees:** Easy to interpret; implicitly select features
- **Random forest:** Ensemble method; little parameter tuning; doesn't overfit as easily as decision trees; attractive and performant (easy parallelization) for many practical problems
- **K-NN:** Lazy learner; predictions can be expensive; good in low dimensions; susceptible to overfitting in high dimensions

Tour of Classifiers (Scikit-learn)



Supervised learning and classification in Scikit-learn:

- **Perceptron:** `from sklearn.linear_model import Perceptron`
- **Logistic regression:** `from sklearn.linear_model import LogisticRegression`
- **Support vector machines:** `from sklearn.svm import SVC`
- **Decision trees:** `from sklearn.tree import DecisionTreeClassifier`
- **Random forest:** `from sklearn.ensemble import RandomForestClassifier`
- **K-NN:** `from sklearn.neighbors import KNeighborsClassifier`

Tour of Classifiers



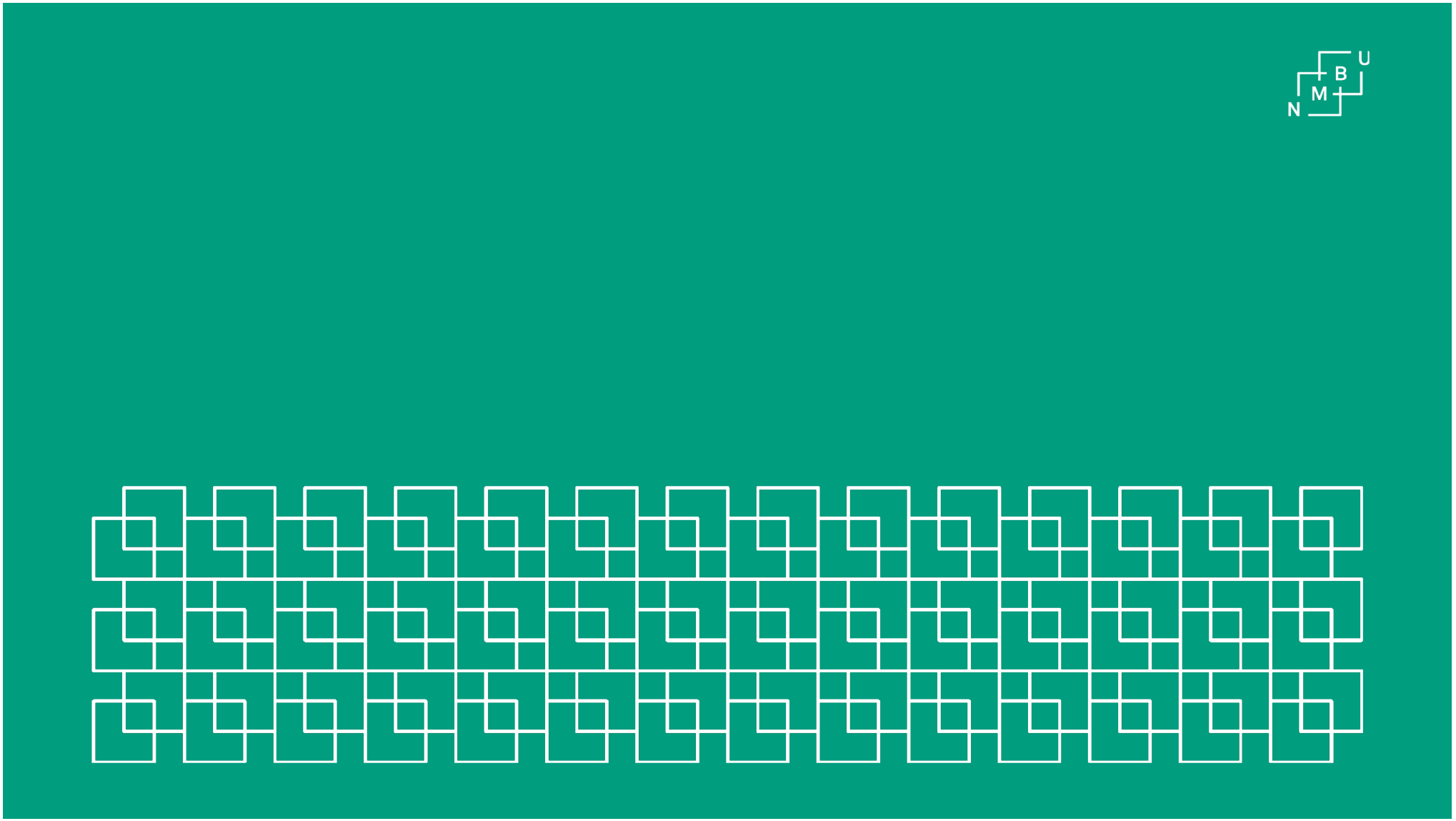
- Apply **several** classifiers to the problem at hand – **no classifier is superior** in **all** situations for **all types** of data (*no-free-lunch theorem*)
- Importance of **training data → no algorithm** will be able to make **good predictions without informative** and **discriminatory features**

  → Next topic in class: Building a good dataset – Pre-processing and Feature selection

Exercise – Comparing different classifiers



- Use the wine dataset from sci-kit learn, and select two features: alcohol and hue
- Split the data into a training set and a test set, using 30% of the data in the test set
- Standardise the data (but use the original scale for Decision tree and Random Forest)
- Compare the following classifiers (with the given parameter settings that are already found using hyperparameter optimisation) in terms of the **number of misclassified samples**, the **accuracy** and the resulting **decision boundaries**:
  - **Perceptron:** eta0=0.0001, max_iter=1
  - **Logistic regression:** C=10
  - **SVC:** C=10, gamma=0.1, kernel='rbf'
  - **Decision tree:** criterion='entropy', max_depth=3, min_samples_split=2
  - **Random Forest:** criterion='gini', max_depth=6, n_estimators=10
  - **K-NN:** n_neighbors=10, p=1


N
M
B
U